

Real-Time Group Chat Application

Using WebSockets

Course: CS559 – Computer System Design
Lab Assignment: WebSockets & Real-Time
Communication

Submitted By (Group Head)

Rohit Raghuwanshi
Roll Number: 12341820

Group Members

S.No.	Name	Roll Number
1	Kabeer Vijay More (Group Head)	12341030
2	Lakshay Gupta	12341300
3	Rathod Chetan Kumar	12341750
4	Rohit Raghuwanshi	12341820

Instructor: Dr. Gagan Raj Gupta

Contents

1 Objective	2
2 Technology Stack	2
3 System Architecture	2
4 Why WebSockets (and not normal HTTP)?	4
5 Deployment Approach: Why We Use ngrok	4
6 Features Implemented	5
7 Commands Used, and Why	5
7.1 Connecting to the Lab Machine	5
7.2 Python Environment Setup	6
7.3 Editing Files on the Remote Machine	6
7.4 Running the Server	6
7.5 Making the Server Reachable from Outside the Lab	7
7.6 Useful Verification Commands	7
8 Backend Source Code	7
8.1 Code Walkthrough	9
9 Frontend (Client Interface)	9
10 Client URL for Testing	11
11 Testing and Screenshots	11
11.1 Two Users Chatting (Core Requirement – Step 1)	12
11.2 A Third User Joining the Same Room (Step 2)	13
11.3 All Four Group Members Connected Simultaneously	14
11.4 Usernames Displayed with Each Message	15
11.5 Timestamps Displayed with Each Message (Bonus)	16
11.6 Join / Leave Notifications	17
11.7 Separate Chat Rooms (Bonus)	18
11.8 Graceful Disconnection Handling	18
11.9 ngrok Running on the Lab Machine	19
11.10 Server Terminal Log	19
12 Observations	20
13 Conclusion	20
14 Note on Use of AI Tools	21

1 Objective

The goal of this assignment is to understand how WebSockets work by building a simple, real-time group chat application. In plain terms: normal websites only talk to a server when you click something (a request goes out, a response comes back, and the connection closes). A WebSocket is different – once it is opened, it stays open, and both sides (the browser and the server) can send messages to each other at any time, instantly, without asking again and again. This is exactly what makes live chat apps, multiplayer games, and live notifications possible.

For this assignment we built:

- One backend server (written in Python) that all clients connect to.
- One frontend web page (HTML, CSS, JavaScript) that any of the four group members can open on their own laptop.
- A shared chat room where everyone who joins can see everyone else's messages instantly.

2 Technology Stack

Component	Technology Used
Backend Server	Python 3, <code>aiohttp</code> (handles both the web page and the WebSocket connections in one process)
Frontend	Plain HTML, CSS, and JavaScript (no framework – kept simple on purpose)
Communication Protocol	WebSocket (<code>ws://</code> / <code>wss://</code>)
Remote Machine Access	SSH (Secure Shell)
Public Exposure	ngrok (creates a public internet link for our server)
Testing Tool	Web browser (multiple tabs / multiple devices)

3 System Architecture

There is exactly **one** backend server running on our allotted lab machine. All four group members' browsers connect to this same server using a WebSocket. Whenever any one person sends a message, the server receives it and immediately re-sends ("broadcasts") it to every other connected browser. Nobody talks to each other directly – everything passes through the one central server, which is what makes it simple to keep everyone in sync.

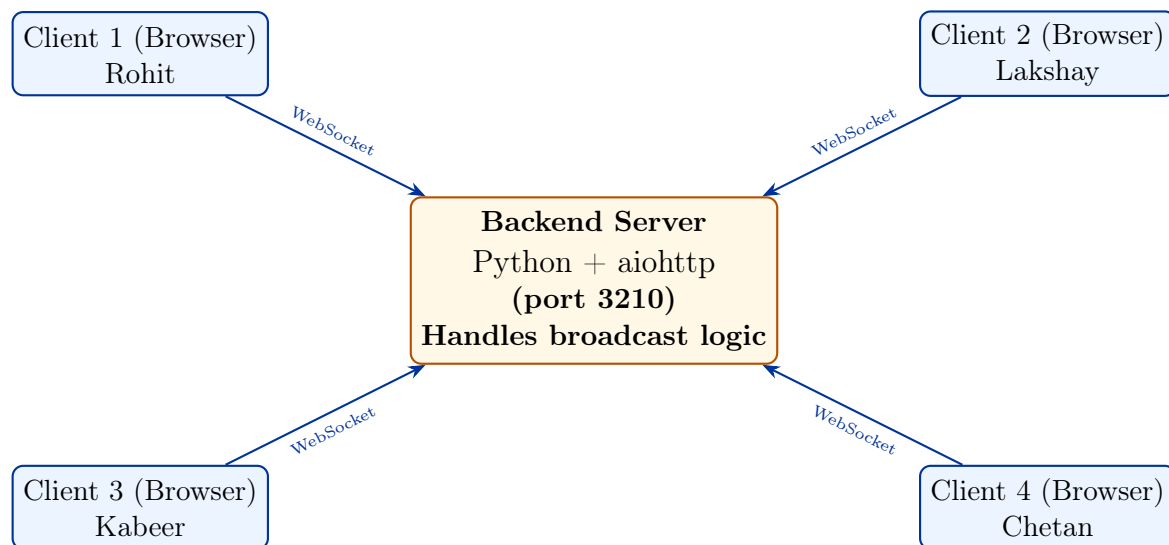


Figure 1: Architecture: one central server broadcasting messages to four connected clients

How a message travels (step by step)

1. A user opens the web page in their browser and enters a username.
2. The browser opens a WebSocket connection to the server (this is called the *handshake* – a one-time setup step after which the connection stays open).
3. The server adds this connection to a list of “connected clients.”
4. When the user types a message and hits Send, the browser sends that text through the open WebSocket.
5. The server receives it, attaches the sender’s username and a timestamp, and forwards (*broadcasts*) it to every other client in the list.
6. Every other browser receives the message instantly and displays it, without ever reloading the page.
7. If a user closes their tab or loses connection, the server detects this, removes them from the list, and tells everyone else that they left.

4 Why WebSockets (and not normal HTTP)?

WebSockets

Normal web browsing uses HTTP: your browser asks a question (request), the server answers once (response), and the connection closes. If you wanted a chat app this way, your browser would have to keep asking the server “any new messages? any new messages?” over and over, which is slow and wasteful.

A WebSocket connection is opened **once** and then stays open in both directions. The server can push a new message to your browser the instant it arrives, without you ever asking for it. This is why WebSockets are the standard choice for chat apps, live sports scores, stock tickers, and multiplayer games.

5 Deployment Approach: Why We Use ngrok

Why ngrok?

A firewall that blocks *incoming* connections on most ports usually still allows *outgoing* connections freely. **ngrok** takes advantage of exactly this: it runs on our lab machine and opens an outgoing connection to ngrok’s own servers on the internet. Anyone who then visits the public link ngrok gives us is routed back through that same connection to our machine. No incoming port ever needs to be opened on the lab machine’s firewall for this to work.

This is not a workaround we are trying to hide – our TA confirmed in writing that *“you can use ngrok provided the underlying code is still hosted on your allotted system,”* and our code genuinely is hosted and running entirely on our allotted lab machine. ngrok only provides the public entry point; it does not run any part of our chat logic itself.

Final Setup (What We Actually Submitted)

In our final setup, both the chat server **and** ngrok run directly on our allotted lab machine. This means the chat stays reachable as long as the lab machine itself is on and our two commands (the server and ngrok) are running – it does not depend on any one person’s laptop staying open.

6 Features Implemented

Requirement	How It Was Implemented
User join notifications	When a WebSocket connection opens, the server broadcasts a system message: " <name> joined the room " to everyone already connected.
User leave notifications	A Python <code>try/finally</code> block guarantees that whenever a connection closes – for <i>any</i> reason – the server removes the user and broadcasts " <name> left the room ".
Real-time broadcasting	Every incoming message is immediately forwarded to all other connected clients using an <code>async</code> loop over the list of active connections.
Username	Each client sends their chosen name as part of the connection URL (<code>?name=...</code>); the server attaches it to every message they send.
Graceful disconnection handling	The <code>try/finally</code> structure ensures that even an abrupt disconnect (closed tab, lost WiFi, browser crash) does not crash the server or affect other users – the disconnected client is simply removed.
Bonus features implemented	
Timestamps	Every message is tagged with the server's current time (HH:MM:SS) when it is broadcast.
Separate chat rooms	Users choose a room name when joining; the server keeps a separate list of connections per room, so messages only broadcast within the same room.

7 Commands Used, and Why

This section lists every command used while setting up and running this project, in the order they were used, with a plain-English explanation of what each one does.

7.1 Connecting to the Lab Machine

SSH Connection

```
ssh -p 2301 student@10.1.75.51
```

Why: SSH (Secure Shell) lets us log into our allotted remote Linux machine securely from our own laptop. `-p 2301` tells it to use our specific assigned port instead of the default port 22.

7.2 Python Environment Setup

Activating Conda

```
conda init
source ~/.bashrc
conda activate base
```

Why: The lab machine uses conda to manage Python packages. `conda init` sets it up for our shell, `source ~/.bashrc` reloads the shell so the change takes effect immediately, and `conda activate base` switches into the base Python environment where all required libraries are available.

Installing the WebSocket Library

```
pip install aiohttp
```

Why: `aiohttp` is a Python library that lets a single program act as both a normal web server (serving our HTML page) and a WebSocket server (handling live chat messages) at the same time.

7.3 Editing Files on the Remote Machine

nano Text Editor

```
nano combined_server.py
```

Why: `nano` is a simple command-line text editor built into Linux. Since the lab machine has no graphical desktop, we used `nano` to write and edit our Python server code directly over SSH. Files are saved with `Ctrl+O`, then `Enter`, and closed with `Ctrl+X`.

7.4 Running the Server

Starting the Backend

```
python3 combined_server.py
```

Why: This starts our WebSocket + web server on the lab machine, listening on port 3210, and keeps it running so clients can connect. We leave this terminal window open the whole time – closing it stops the chat for everyone.

7.5 Making the Server Reachable from Outside the Lab

Installing ngrok on the Lab Machine

```
wget https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-amd64.tgz
tar -xvzf ngrok-v3-stable-linux-amd64.tgz
./ngrok config add-authtoken <our-authtoken>
```

Why: We download and set up ngrok directly on our allotted lab machine (not on our personal laptops), so the public link keeps working even if a group member's laptop is closed. The authtoken links this ngrok installation to our free ngrok account, which is required before it can create a tunnel.

Starting the Public Tunnel

```
./ngrok http 3210
```

Why: This is the command that actually creates the public HTTPS link (e.g. <https://dipping-quizzical-ellipse.ngrok-free.dev>) and forwards any traffic sent to it straight to our server on port 3210, on the same machine. This is the exact link we share with teammates and TAs. We ran this in a second SSH terminal, separate from the one running the server, so both keep running side by side.

7.6 Useful Verification Commands

Command	Purpose
<code>pwd</code>	Shows current directory – used to confirm whether we were on the local laptop or the remote machine.
<code>ls -la</code>	Lists all files (including hidden ones) in the current folder, to confirm downloads/extractions succeeded.
<code>uname -m</code>	Shows CPU architecture (e.g. <code>arm64</code>) – needed to download the correct ngrok binary for our laptop.
<code>curl http://localhost:PORT</code>	Tests whether a server is reachable and responding, directly from the command line, without needing a browser.
<code>Ctrl+C</code>	Stops a currently running program (e.g. to stop the server before editing and restarting it).

8 Backend Source Code

The full backend server is a single Python file, `combined_server.py`. It does two jobs at once: (1) serves the HTML/CSS/JavaScript frontend when a browser visits the link, and (2) handles the live WebSocket chat connections and message broadcasting.

Listing 1: combined_server.py – full backend source code

```

1 from aiohttp import web, WSMsgType
2 from datetime import datetime
3
4 # rooms = { room_name: { websocket: username } }
5 rooms = {}
6
7 async def index(request):
8     # Serves the frontend HTML page (see Section 9 for full HTML)
9     return web.Response(text=HTML_PAGE, content_type="text/html")
10
11 async def broadcast(room, payload, exclude=None):
12     dead_clients = []
13     for client in list(rooms.get(room, {})):
14         if client != exclude:
15             try:
16                 await client.send_json(payload)
17             except Exception:
18                 # This client's connection is already closing/broken.
19                 # Mark it for cleanup instead of crashing the broadcast.
20                 dead_clients.append(client)
21
22     # Remove any clients that failed to receive the message
23     for dead in dead_clients:
24         if dead in rooms.get(room, {}):
25             del rooms[room][dead]
26
27 async def websocket_handler(request):
28     ws = web.WebSocketResponse()
29     await ws.prepare(request)
30
31     username = request.query.get("name", "Anonymous")
32     room = request.query.get("room", "general")
33
34     if room not in rooms:
35         rooms[room] = {}
36     rooms[room][ws] = username
37
38     print(f"[{room}] {username} joined. Total in room: {len(rooms[room])}")
39
40     # Notify everyone in the room that a new user joined
41     await broadcast(room, {
42         "type": "system",
43         "text": f"{username} joined the room"
44     })
45
46     try:
47         async for msg in ws:
48             if msg.type == WSMsgType.TEXT:
49                 timestamp = datetime.now().strftime("%H:%M:%S")
50                 payload = {
51                     "type": "message",
52                     "user": username,
53                     "text": msg.data,
54                     "time": timestamp
55                 }
56                 print(f"[{room}] {username} ({timestamp}): {msg.data}")
57                 await broadcast(room, payload)
58     finally:
59         # Runs no matter how the connection ends -- graceful disconnect handling
60         if room in rooms and ws in rooms[room]:
61             del rooms[room][ws]
62             print(f"[{room}] {username} left. Total in room: {len(rooms[room])}")
63             await broadcast(room, {
64                 "type": "system",
65                 "text": f"{username} left the room"

```

```
66         })
67
68     return ws
69
70 app = web.Application()
71 app.router.add_get("/", index)
72 app.router.add_get("/ws", websocket_handler)
73
74 if __name__ == "__main__":
75     web.run_app(app, host="0.0.0.0", port=3210)
```

8.1 Code Walkthrough

- **rooms dictionary:** keeps track of every connected client, grouped by which chat room they picked. This is how we support the "separate chat rooms" bonus feature.
- **websocket_handler:** runs once for every new client that connects. It reads their chosen name and room from the connection URL.
- **broadcast function:** loops through every client currently in the same room and sends them the message – this is the core of "real-time broadcasting." It is written as a standalone function (rather than nested inside `websocket_handler`) so it can be reused cleanly wherever it's needed.
- **Handling clients that disconnect mid-broadcast:** during testing with multiple users, we found that if one person's tab closed at the exact moment a message was being broadcast, the server would throw a `ClientConnectionResetError` trying to write to that closing connection. We fixed this by wrapping each send in a `try/except`: if sending to a client fails, that client is simply marked and removed from the room instead of stopping the whole broadcast. This is what makes disconnection handling truly graceful, even under real multi-user conditions, not just when one user leaves at a time.
- **try / finally:** the most important part for "graceful disconnection handling" of the connection itself. Whatever happens inside the `try` block (messages being sent back and forth), the `finally` block is *guaranteed* to run when the connection ends, so we never leave a "ghost" user in the room, and the server never crashes because of one bad disconnect.
- **timestamp:** we grab the current server time every time a message is sent, satisfying the timestamp bonus feature.

9 Frontend (Client Interface)

The frontend is plain HTML, CSS, and JavaScript, served directly by the backend at the root URL (/). No framework was used, in line with the assignment's instruction that the interface "need not be feature rich but should provide a usable interface."

Design Choices

- A simple “Join Chat” screen asks for a username and a room name before entering the chat.
- Messages appear as chat bubbles – your own messages align to the right in a highlighted color, other people’s messages align to the left, similar to familiar messaging apps.
- System messages (joins/leaves) are shown as small centered grey text, visually distinct from real chat messages.
- Each message shows the sender’s name and the time it was sent.

Listing 2: Key JavaScript logic connecting the frontend to the backend WebSocket

```
1 const proto = location.protocol === "https:" ? "wss" : "ws";
2 ws = new WebSocket(
3   proto + "://" + location.host +
4   "/ws?name=" + encodeURIComponent(myName) +
5   "&room=" + encodeURIComponent(room)
6 );
7
8 ws.onmessage = (event) => {
9   const data = JSON.parse(event.data);
10  addMessage(data); // renders the message as a chat bubble
11 };
12
13 function send() {
14   const input = document.getElementById("msg");
15   if (input.value.trim() !== "") {
16     ws.send(input.value); // sends the typed message to the server
17     input.value = "";
18   }
19 }
```

Why wss:// instead of ws:// sometimes?

ws:// is a plain (unencrypted) WebSocket connection, while **wss://** is the secure, encrypted version – the WebSocket equivalent of the difference between **http://** and **https://**. Since our final deployment uses ngrok’s HTTPS link (see Section 10), the browser automatically requires **wss://** for the WebSocket part too. Our JavaScript code checks how the page itself was loaded and automatically picks **ws://** or **wss://** accordingly, so the same code works in both setups.

10 Client URL for Testing

Live Testing Link

As explained in Section 5, we tested the direct campus-network URL first, then switched to ngrok, which runs directly on our allotted lab machine. TAs can test the application using the link below – no login, no campus WiFi, and no setup required, since ngrok gives us a normal public internet address:

<https://dipping-quizzical-ellipse.ngrok-free.dev>

When opening the link, ngrok may show a one-time 'Visit Site' confirmation page this is normal for free ngrok accounts. Click 'Visit Site' to proceed to the chat application."
Note: This link only works while our backend server is running on our lab machine. If the link appears to be down at the time of testing, please contact the group and we will restart it.

11 Testing and Screenshots

11.1 Two Users Chatting (Core Requirement – Step 1)

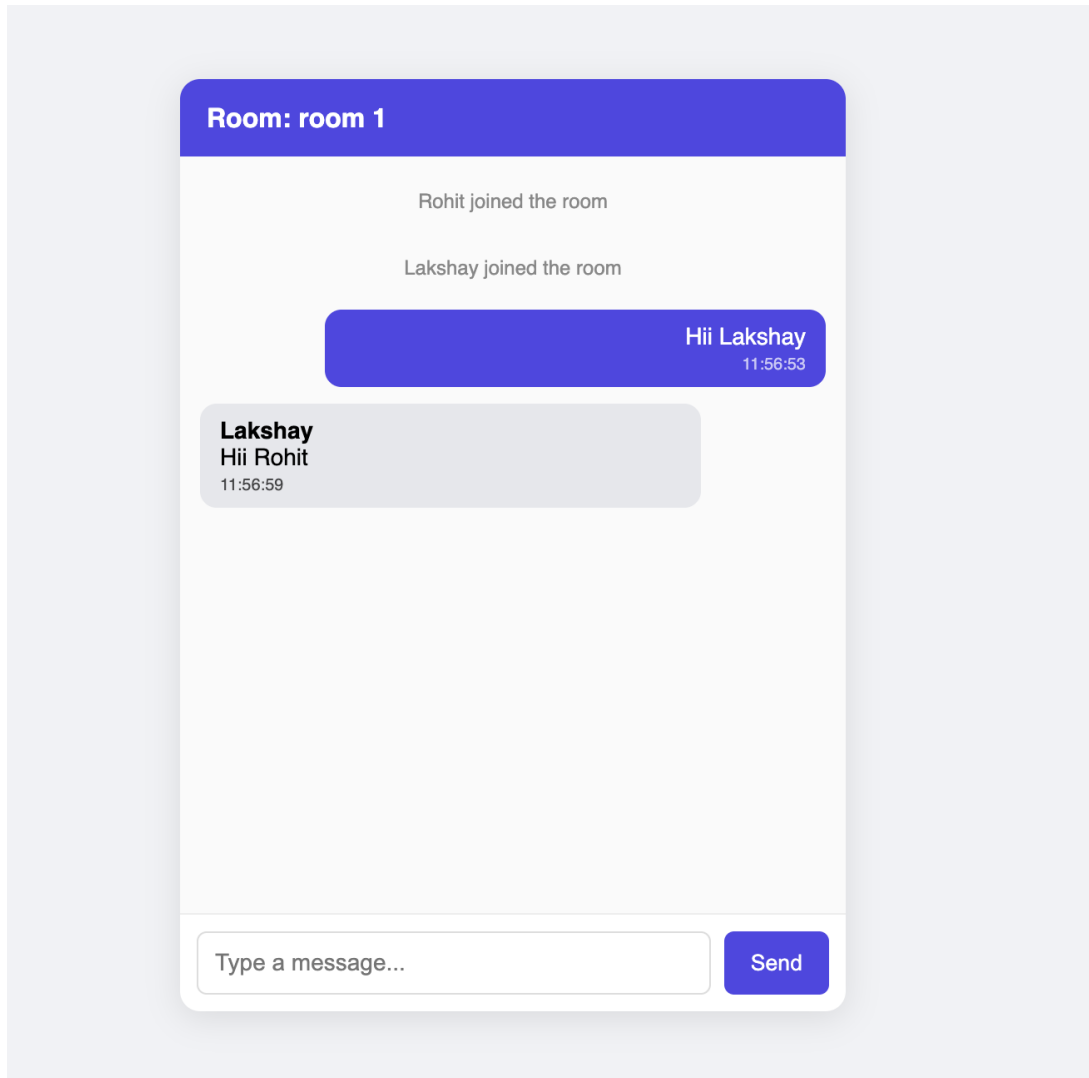


Figure 2: Two users chatting successfully

11.2 A Third User Joining the Same Room (Step 2)

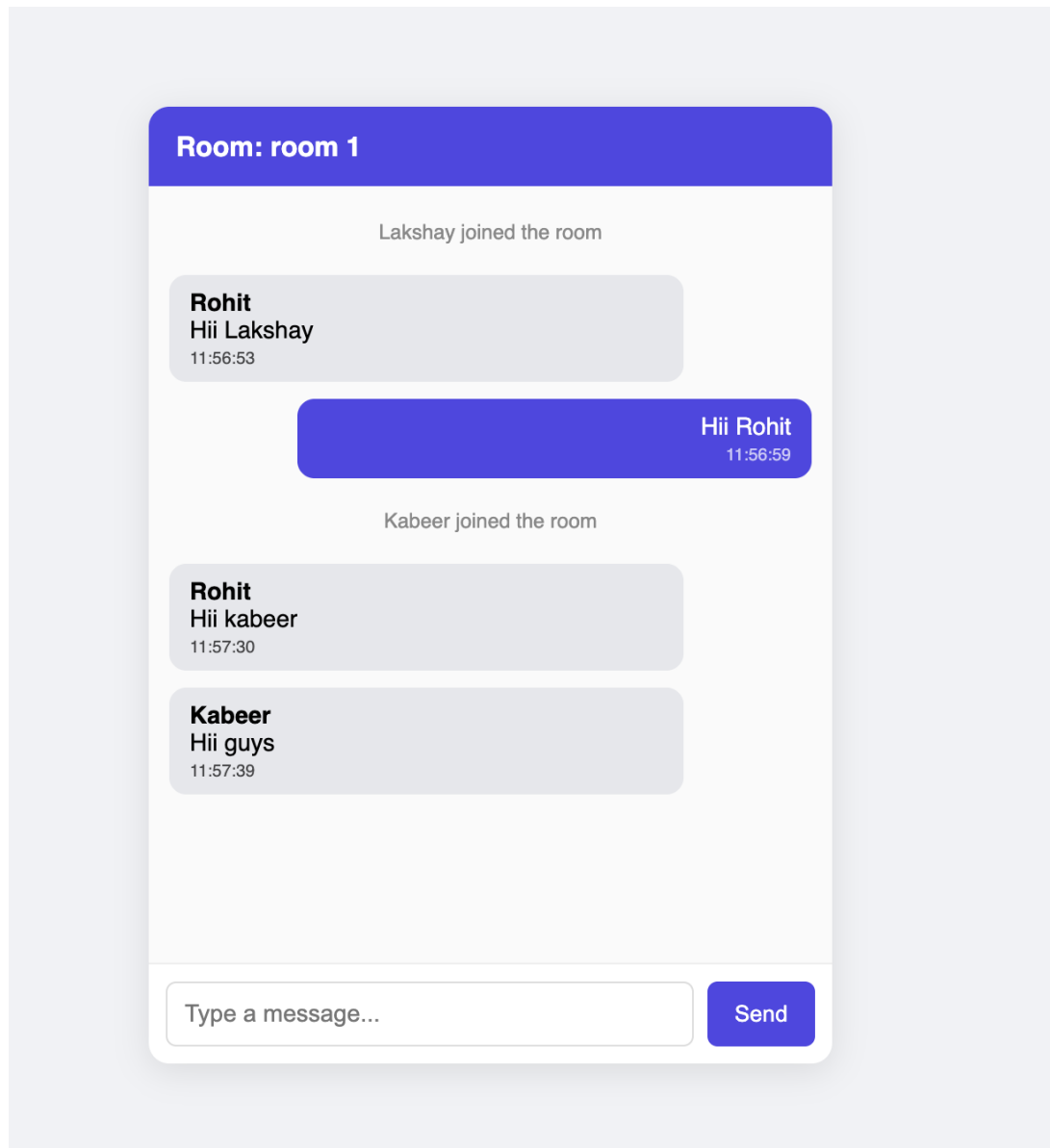


Figure 3: Third user joining and broadcasting confirmed

11.3 All Four Group Members Connected Simultaneously

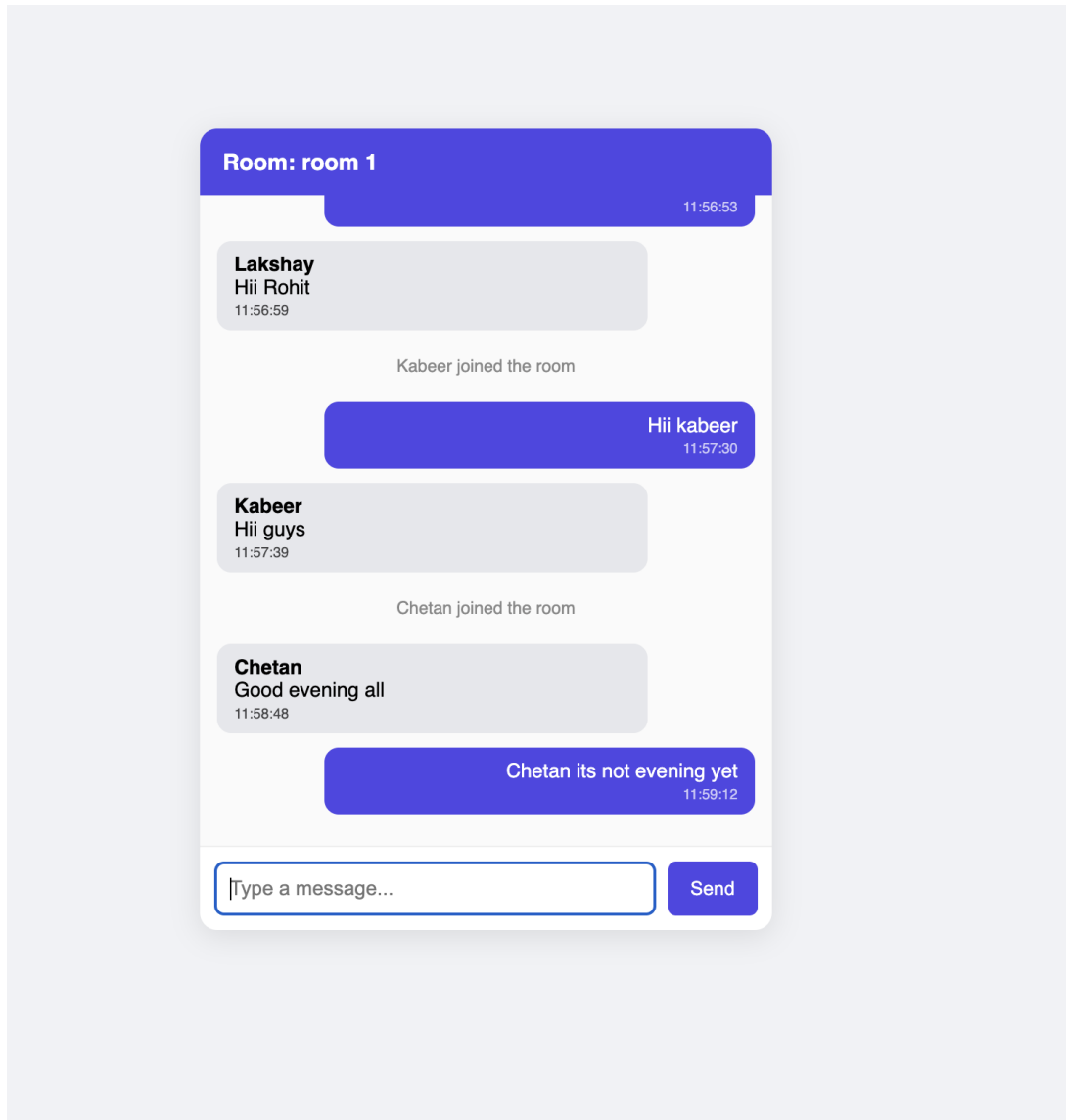


Figure 4: Full four-user group chat session

11.4 Usernames Displayed with Each Message

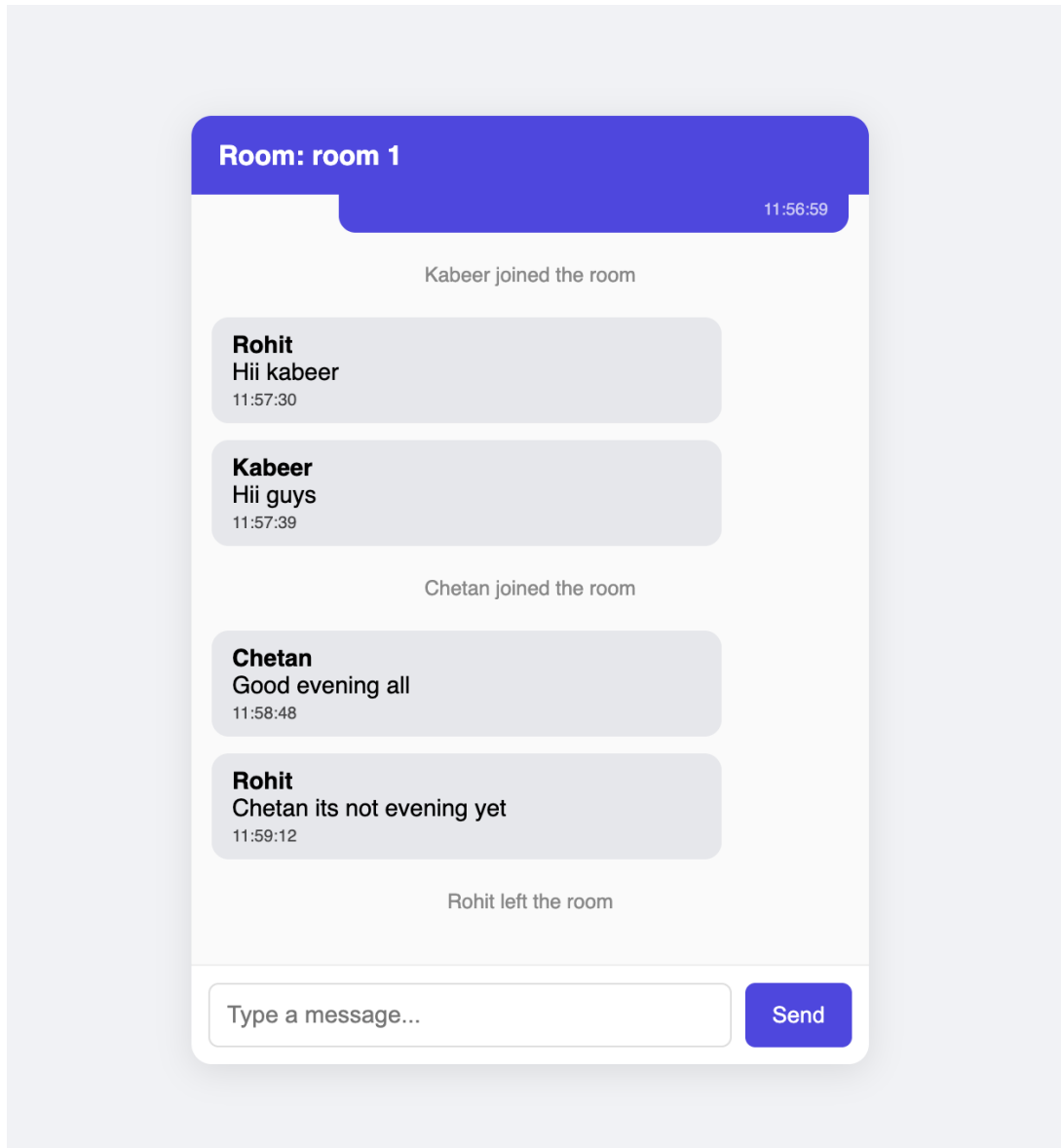


Figure 5: Usernames shown alongside messages

11.5 Timestamps Displayed with Each Message (Bonus)

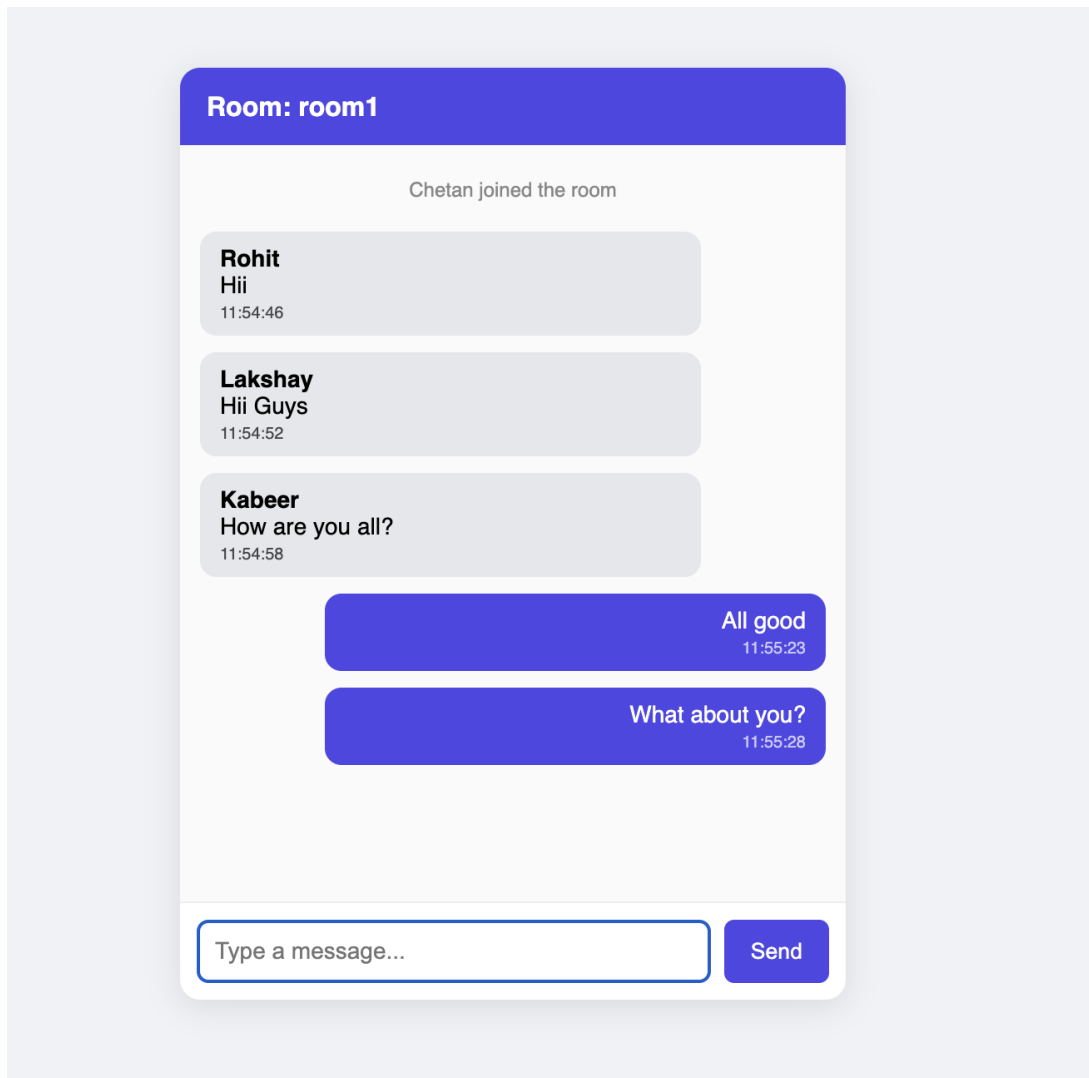


Figure 6: Timestamps displayed with each message

11.6 Join / Leave Notifications

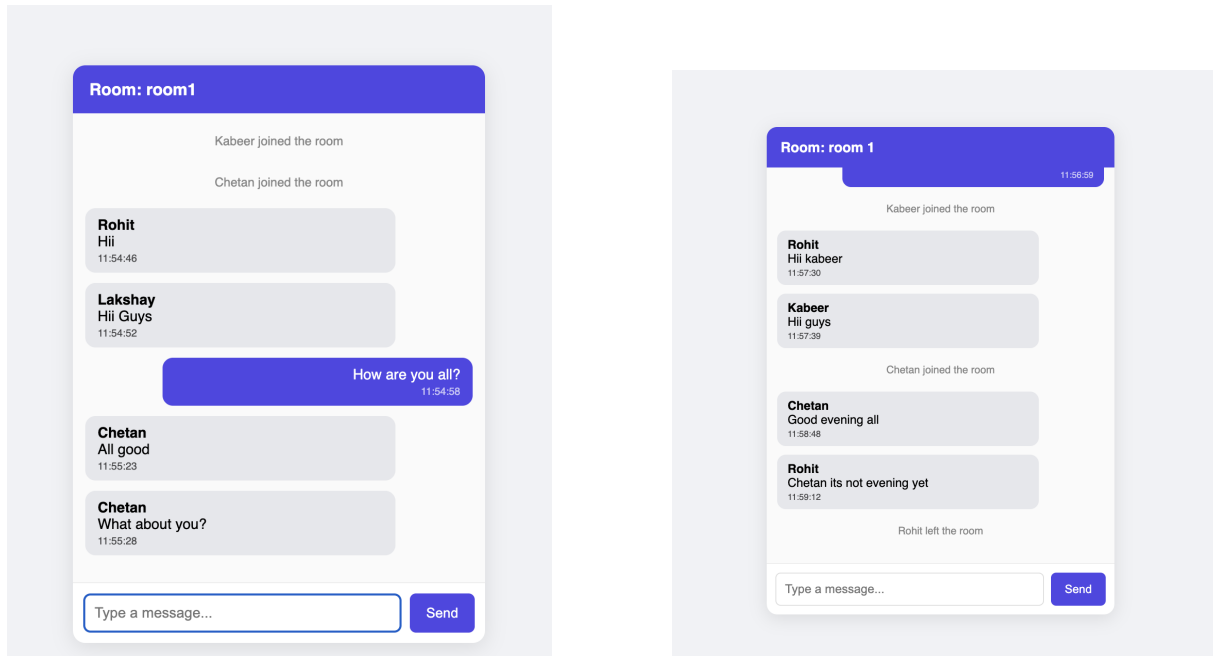


Figure 7: Join and leave notifications in action

11.7 Separate Chat Rooms (Bonus)

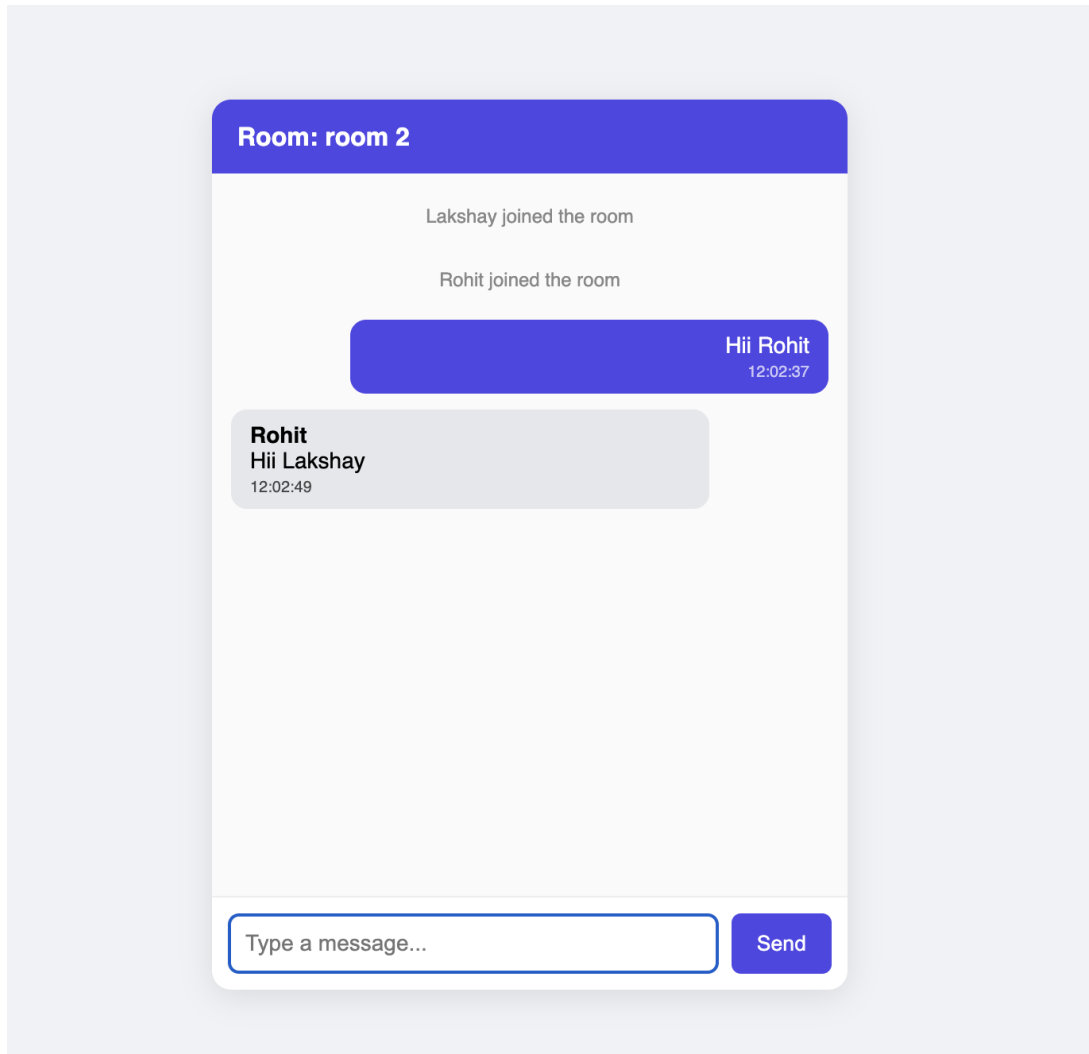


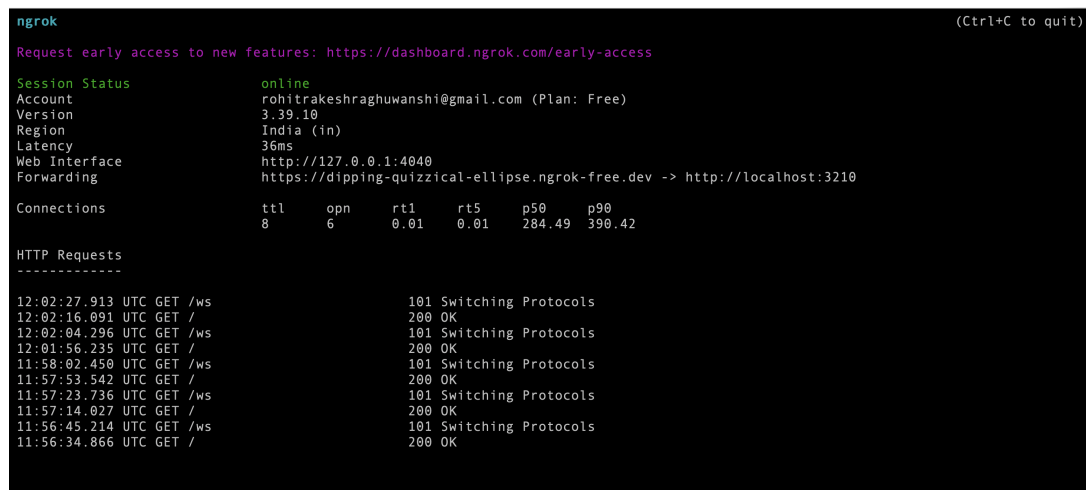
Figure 8: Room separation working correctly

11.8 Graceful Disconnection Handling

```
[room 1] Chetan joined. Total in room: 4
[room 1] Chetan (11:58:48): Good evening all
[room 1] Rohit (11:59:12): Chetan its not evening yet
[room 1] Rohit left. Total in room: 3
```

Figure 9: Server handling a client disconnect gracefully

11.9 ngrok Running on the Lab Machine



```

ngrok (Ctrl+C to quit)
Request early access to new features: https://dashboard.ngrok.com/early-access

Session Status      online
Account             rohitakeshraghuwanshi@gmail.com (Plan: Free)
Version             3.39.10
Region              India (in)
Latency              36ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://dipping-quizzical-ellipse.ngrok-free.dev -> http://localhost:3210

Connections
  ttl   opn   rt1   rt5   p50   p90
    8    6   0.01  0.01  284.49 390.42

HTTP Requests
-----
12:02:27.913 UTC GET /ws      101 Switching Protocols
12:02:16.091 UTC GET /      200 OK
12:02:04.296 UTC GET /ws      101 Switching Protocols
12:01:56.235 UTC GET /      200 OK
11:58:02.450 UTC GET /ws      101 Switching Protocols
11:57:53.542 UTC GET /      200 OK
11:57:23.736 UTC GET /ws      101 Switching Protocols
11:57:14.027 UTC GET /      200 OK
11:56:45.214 UTC GET /ws      101 Switching Protocols
11:56:34.866 UTC GET /      200 OK
  
```

Figure 10: ngrok tunnel active on our allotted lab machine

11.10 Server Terminal Log

```

[student@stu26_sys1:~$ python3 combined_server.py
===== Running on http://0.0.0.0:3210 =====
(Press CTRL+C to quit)
[general] Rohit joined. Total in room: 1
[general] Rohit left. Total in room: 0
[room1] Rohit joined. Total in room: 1
[room1] Lakshay joined. Total in room: 2
[room1] Kabeer joined. Total in room: 3
[room1] Chetan joined. Total in room: 4
[room1] Rohit (11:54:46): Hii
[room1] Lakshay (11:54:52): Hii Guys
[room1] Kabeer (11:54:58): How are you all?
[room1] Chetan (11:55:23): All good
[room1] Chetan (11:55:28): What about you?
[room 1] Rohit joined. Total in room: 1
[room 1] Lakshay joined. Total in room: 2
[room 1] Rohit (11:56:53): Hii Lakshay
[room 1] Lakshay (11:56:59): Hii Rohit
[room 1] Kabeer joined. Total in room: 3
[room 1] Rohit (11:57:30): Hii kabeer
[room 1] Kabeer (11:57:39): Hii guys
  
```

Figure 11: Server-side log of chat activity

“““

12 Observations

Observations & Inferences

- Once a WebSocket connection is established, message delivery between clients felt immediate, with no noticeable delay.
- We initially assumed our own server code might be broken when the direct URL failed in the browser. Testing with `curl` directly on the lab machine was the key step that proved the server was working correctly all along, and that the problem was the campus network's firewall, not our code.
- Using a single `try/finally` block was enough to make disconnect handling fully graceful – we tested this by force-closing browser tabs and even quitting the browser entirely, and in both cases the server correctly removed the client and notified the room without crashing.
- Keeping a per-room dictionary of connected clients, instead of one global list, made adding the "separate chat rooms" bonus feature straightforward with minimal extra code.
- Running ngrok directly on our allotted lab machine (instead of on a personal laptop) meant the public link kept working independent of any one group member's laptop being open, which made testing far more reliable.
- The assignment's emphasis on seeing the "WebSocket handshake, message send/receive, and broadcast in action" was best demonstrated by watching the server's terminal log update live as each group member joined, sent messages, and disconnected.

13 Conclusion

In this assignment, we built a real-time group chat application using WebSockets, with a single Python backend (using `aiohttp`) and a lightweight HTML/CSS/JavaScript frontend. The application successfully supports multiple simultaneous users in a shared chat room, with usernames, timestamps, join/leave notifications, real-time message broadcasting, and graceful handling of client disconnections. We additionally implemented separate chat rooms as a bonus feature. We first attempted to expose our server using a direct URL on the campus network as suggested, but found through testing that the lab network's firewall blocks direct external access to any port other than SSH. We confirmed this with clear evidence (the server worked correctly when tested locally, but was unreachable externally), and used ngrok – running directly on our allotted lab machine – as a justified workaround, consistent with what our TA had approved. This exercise gave us hands-on understanding of how persistent, bidirectional WebSocket connections differ from the traditional request-response model of HTTP, how a central server can coordinate real-time communication between many independent clients, and how real deployments often need

to work around network restrictions that are outside a developer's control.

14 Note on Use of AI Tools

Disclosure

An AI assistant was used during this assignment in a limited and supportive capacity. Its use was primarily for:

1. **Debugging assistance** to help identify and resolve implementation issues.
2. **Report formatting and presentation** to improve the structure and consistency of the documentation.

The application design, implementation, execution, and testing of the chat application were carried out by our group. All technical decisions and final outcomes were independently reviewed and determined by us, including the decision to use **ngrok**, which was made after consultation with our TA.